

Simple exercise questions

1. ArrayList – Sum of Elements

Problem: Given an integer `n` and a list of `n` integers, compute and print their sum.

Input Format:

First line: integer `n` ($1 \leq n \leq 1000$)

Second line: `n` space-separated integers

Output Format:

A single integer – the sum.

Example:

Input:

```
5
1 2 3 4 5
```

Output:

```
15
```

2. ArrayList – Find the Maximum

Problem: Read `n` integers into an `ArrayList<Integer>` and output the maximum value.

Input Format:

First line: `n`

Second line: `n` integers

Output Format:

One integer – the maximum.

Example:

Input:

```
4
10 20 5 30
```

Output:

3. ArrayList – Remove Even Numbers

Problem: Given an `ArrayList<Integer>` of size `n`, remove all even numbers and print the remaining list (preserving order). Use `Iterator` for safe removal.

Input Format:

First line: `n`

Second line: `n` integers

Output Format:

Space-separated odd numbers (if none, print `empty`).

Example:

Input:

```
6
1 2 3 4 5 6
```

Output:

```
1 3 5
```

4. ArrayList – Insert at Position

Problem: You are given an initial list of integers. Then you receive `q` queries. Each query is either:

- `Insert x y` – insert `y` at index `x`
- `Delete x` – remove element at index `x`

After all queries, print the final list.

Input Format:

First line: initial size `n`

Second line: `n` integers

Third line: number of queries `q`

Next `q` lines: each starts with a string (`Insert` or `Delete`)

Output Format:

Space-separated final list

Example:

Input:

```
5
10 20 30 40 50
3
Insert 2 25
Delete 4
Insert 0 5
```

Output:

```
5 10 20 25 40
```

5. LinkedList – Remove First and Last

Problem: Create a `LinkedList<String>` with the names of `n` students. Remove the first and last element, then print the remaining list.

Input Format:

First line: `n`

Next `n` lines: student names

Output Format:

Space-separated names after removal

Example:

Input:

```
4
Alice
Bob
Charlie
David
```

Output:

```
Bob Charlie
```

6. LinkedList – Convert to ArrayList

Problem: Read `n` integers into a `LinkedList<Integer>`, then copy all elements into an `ArrayList<Integer>` (in the same order) and print the `ArrayList`.

Input Format:

First line: `n`

Second line: `n` integers

Output Format:

Space-separated integers (same order)

Example:

Input:

```
3
100 200 300
```

Output:

```
100 200 300
```

7. Stack – Reverse a String

Problem: Use a `Stack<Character>` (or `ArrayDeque` as stack) to reverse a given string.

Input Format:

A single line containing a string `s` ($1 \leq |s| \leq 1000$)

Output Format:

The reversed string

Example:

Input:

```
hello
```

Output:

```
olleh
```

8. Stack – Check Balanced Parentheses

Problem: Given a string containing only `(`, `)`, `{`, `}`, `[`, `]`, determine if it is balanced.

Problem: Given a string containing only '{', '}', '(', ')', '[', ']', determine if it is balanced.
Use a stack.

Input Format:

First line: number of test cases `T`

Next `T` lines: each a bracket string

Output Format:

For each test case, print `YES` if balanced else `NO`

Example:

Input:

```
2
{[()]}
```

Output:

```
YES
NO
```

9. Queue – Print in FIFO Order

Problem: Read `n` integers into a `Queue<Integer>` (use `ArrayDeque` or `LinkedList`). Then dequeue all elements and print them in the order they were removed.

Input Format:

First line: `n`

Second line: `n` integers

Output Format:

Space-separated integers (same order as input)

Example:

Input:

```
4
1 2 3 4
```

Output:

```
1 2 3 4
```

10. Queue – Hot Potato (Josephus)

Problem: n people (numbered $1..n$) stand in a queue. Every k -th person (counting from the front) is removed and printed. Continue until the queue is empty. Simulate using a `Queue<Integer>`.

Input Format:

Two integers n and k ($1 \leq n, k \leq 100$)

Output Format:

Space-separated removal order

Example:

Input:

```
5 2
```

Output:

```
2 4 1 5 3
```

(Explanation: start 1,2,3,4,5 → remove 2 → 3,4,5,1 → remove 4 → 5,1,3 → remove 1 → 3,5 → remove 5 → 3 → remove 3)

11. Queue – Recent Calls (Rate Limiter)

Problem: You have a queue that stores timestamps (in seconds) of requests. When a new request arrives at time t , remove all requests older than $t-3000$ (i.e., keep only the last 5 minutes). Print the size of the queue after each new request.

Input Format:

First line: number of requests n

Second line: n space-separated integers (timestamps in increasing order)

Output Format:

For each request, print the queue size after cleaning

Example:

Input:

```
6
1 100 3001 3002 6000 7000
```

Output:

```
1 2 3 3 2 1
```

12. Iterator – Remove All Occurrences of a Value

Problem: Given an `ArrayList<String>` and a target string, use an `Iterator` to remove **all** occurrences of the target. Print the remaining list.

Input Format:

First line: size `n`

Next `n` lines: strings

Last line: target string

Output Format:

Space-separated strings after removal (order preserved)

Example:

Input:

```
5
apple
banana
apple
orange
apple
apple
```

Output:

```
banana orange
```

13. ListIterator – Replace Every Second Element

Problem: Use a `ListIterator` to traverse a `LinkedList<Integer>` and replace each element at an **even index** (0-based) with its negative value. Print the modified list.

Input Format:

First line: `n`

Second line: `n` integers

Output Format:

Space-separated integers after modification

Example:

Input:

```
5
10 20 30 40 50
```

Output:

```
-10 20 -30 40 -50
```

14. Vector – Legacy Operations

Problem: Create a `Vector<String>` . Add three elements: `"Java"` , `"Collections"` , `"Vector"` .

- Insert `"Legacy"` at index 1.
 - Replace the element at index 2 with `"Framework"` .
 - Remove the last element.
- Print the final vector.

Output Format:

Space-separated elements

Example Output:

```
Java Legacy Framework
```

15. Stack + Queue – Palindrome Checker

Problem: Use both a stack and a queue to check if a given string (ignoring case and non-letters) is a palindrome.

- Push each character to a stack and add to a queue.
 - Then pop from stack and poll from queue, comparing the characters.
- Print `YES` if palindrome else `NO` .

Input Format:

A single line containing a string (may contain spaces and punctuation)

Output Format:

Output format.

YES or **NO**

Example:

Input:

A man, a plan, a canal: Panama

Output:

YES